Vistas en Bases de Datos

Teoría, DER unificado y ejemplos autocontenidos

¿Qué es una vista?

Una **vista** es una tabla virtual definida por una consulta `SELECT` almacenada. No guarda datos físicamente; cada vez que se consulta, MySQL ejecuta la consulta asociada y devuelve el resultado en tiempo real. Es una capa de abstracción que simplifica consultas complejas, mejora la seguridad y facilita el mantenimiento.

📐 Esquema de tablas utilizado en todos los ejemplos

Para que los ejemplos sean coherentes y fáciles de entender, utilizaremos las siguientes tablas relacionadas. Este modelo refleja un escenario de comercio con clientes, productos y ventas.

```
Tablas y relaciones:
clientes          productos          ventas
-----------------  -----------------  ----------------------
id_cliente (PK)    id_producto (PK)   id_venta (PK)
nombre             nombre             cliente_id (FK → clientes)
apellido           precio             producto_id (FK → productos)
email              stock              cantidad
ciudad                                fecha_venta
estado (activo/inactivo)

Relaciones:
- Un cliente puede tener muchas ventas (1:N)
- Un producto puede aparecer en muchas ventas (1:N)
- La tabla ventas registra cada compra con cantidad y fecha.

Nota: En los ejemplos se usarán estas mismas tablas, adaptando nombres según el caso (por ejemplo, clientes, ventas, productos).
```

🔄 Vistas actualizables: qué son y cómo funcionan en MySQL

Una vista es **actualizable** cuando permite operaciones `INSERT`, `UPDATE` y `DELETE` sobre ella. MySQL sí soporta vistas actualizables, pero solo si cumplen estrictas condiciones. En caso contrario, la vista es de solo lectura.

Requisitos para que una vista sea actualizable en MySQL:

- Debe basarse en una única tabla (no puede tener JOIN, UNION, subconsultas correlacionadas).
- No puede contener funciones de agregación (`SUM`, `COUNT`, `AVG`, `MAX`, `MIN`).
- No puede contener `GROUP BY`, `HAVING`, `DISTINCT`, `LIMIT` .
- No puede incluir expresiones o literales en la lista `SELECT` que no sean directamente columnas de la tabla base.
- No puede tener subconsultas en el `SELECT` ni en la cláusula `WHERE` que hagan referencia a la misma tabla.

Ejemplo práctico: vista actualizable sobre la tabla clientes

```
-- Tabla base
CREATE TABLE clientes (
    id_cliente INT PRIMARY KEY AUTO_INCREMENT,
    nombre VARCHAR(50),
    apellido VARCHAR(50),
    email VARCHAR(100),
    ciudad VARCHAR(50),
    activo BOOLEAN DEFAULT TRUE
);

-- Vista simple y actualizable (solo muestra datos básicos)
CREATE VIEW v_clientes_activos AS
SELECT id_cliente, nombre, apellido, email
FROM clientes
WHERE activo = TRUE;

-- Operaciones permitidas:
UPDATE v_clientes_activos SET email = 'nuevo@mail.com' WHERE id_cliente = 10;
INSERT INTO v_clientes_activos (nombre, apellido, email) VALUES ('Lucía', 'Méndez', 'lucia@mail.com');
DELETE FROM v_clientes_activos WHERE id_cliente = 5;  -- solo si no viola FK
```

En este caso, la vista cumple todos los requisitos: una sola tabla, sin agregaciones, sin literales en el SELECT (todas son columnas reales de `clientes`). MySQL permite modificaciones, que se reflejarán directamente en la tabla `clientes`.

Importante: aunque la vista tenga `WHERE activo = TRUE`, una inserción a través de ella no incluye el campo `activo`, por lo que se asignará el valor por defecto (`TRUE`) y la fila aparecerá en la vista. Si se quisiera restringir inserciones que no cumplan la condición, se debe usar `WITH CHECK OPTION`.

¿Qué pasa si intentamos actualizar una vista no actualizable?
MySQL devuelve un error como `"ER_NON_UPDATABLE_TABLE"` (Tabla no actualizable). Por ejemplo, si la vista contiene un JOIN o una función agregada, cualquier operación DML fallará.

📦 Vistas materializadas: concepto y alternativas en MySQL

Una **vista materializada** es una vista que **almacena físicamente** el resultado de la consulta, como si fuera una tabla. Esto mejora drásticamente el rendimiento en consultas pesadas (agregaciones, joins complejos) porque el resultado ya está precalculado y se puede indexar. Sin embargo, requiere mecanismos para refrescar los datos cuando cambian las tablas base.

MySQL **no soporta vistas materializadas de forma nativa** (a diferencia de Oracle o PostgreSQL). En su lugar, se pueden simular mediante:

- **Tablas temporales** que se recargan periódicamente (por ejemplo, con eventos programados).
- **Triggers** que actualicen una tabla de resumen cuando se modifican las tablas base.
- Uso de motores de almacenamiento como `ARCHIVE` o `MyISAM` combinado con procedimientos almacenados.

Ejemplo conceptual de simulación:
Crear una tabla `resumen_ventas_mensuales` y un trigger que la actualice cada vez que se inserta una venta. Aunque no es una vista materializada automática, cumple un propósito similar.

📝 ¿Qué significa "literales" en el contexto de vistas actualizables?

En SQL, un **literal** es un valor fijo escrito directamente en la consulta, como un número (`100`), una cadena (`'Madrid'`), una fecha (`'2025-03-20'`) o una constante booleana. Cuando una vista incluye un literal en su `SELECT` (por ejemplo, `SELECT nombre, 'cliente' AS tipo`), esa columna no se corresponde con ninguna columna de la tabla base. Por lo tanto, al intentar actualizar la vista, MySQL no sabría cómo modificar ese valor en la tabla subyacente, y la vista deja de ser actualizable.

Ejemplo con literal que impide actualización

```
-- Vista con literal
CREATE VIEW v_clientes_con_tipo AS
SELECT id_cliente, nombre, 'ACTIVO' AS estado_vista
FROM clientes;

-- Intento de UPDATE falla porque 'estado_vista' no es una columna real
UPDATE v_clientes_con_tipo SET estado_vista = 'INACTIVO' WHERE id_cliente = 2;  -- ❌ Error
```

En cambio, si la vista solo usa columnas reales (sin literales, funciones que no sean directamente asignables), puede ser actualizable.

🛡️ WITH CHECK OPTION: asegurando que las modificaciones respeten la condición

Cuando una vista es actualizable y tiene una cláusula `WHERE`, podemos usar `WITH CHECK OPTION` para impedir que se inserten o actualicen filas que queden fuera del filtro de la vista. Es decir, garantiza que todas las filas manipuladas a través de la vista sigan cumpliendo la condición de la vista.

Utilizando nuestras tablas `clientes` (con ciudad y activo), mostramos ejemplos claros:

Vista con CHECK OPTION (sin incluir la columna filtrada)

```
CREATE VIEW v_clientes_madrid AS
SELECT id_cliente, nombre, email
FROM clientes
WHERE ciudad = 'Madrid'
WITH CHECK OPTION;

-- Ahora sí podemos insertar siempre que ciudad='Madrid'
INSERT INTO v_clientes_madrid_v2 (id_cliente, nombre, email, ciudad)
VALUES (51, 'Ana', 'ana@mail.com', 'Madrid');  -- ✅ Correcto

-- Esta inserción falla porque ciudad no es Madrid
INSERT INTO v_clientes_madrid_v2 (id_cliente, nombre, email, ciudad)
VALUES (52, 'Luis', 'luis@mail.com', 'Barcelona'); -- ❌ Fallo CHECK OPTION
```

Motivo: la vista no expone la columna `ciudad`, por lo que MySQL no puede garantizar que la nueva fila cumpla `ciudad='Madrid'`. La operación es rechazada.

Vista con CHECK OPTION (incluyendo la columna filtrada)

```
CREATE VIEW v_clientes_madrid_v2 AS
SELECT id_cliente, nombre, email, ciudad
FROM clientes
WHERE ciudad = 'Madrid'
WITH CHECK OPTION;

-- Ahora sí podemos insertar siempre que ciudad='Madrid'
INSERT INTO v_clientes_madrid_v2 (id_cliente, nombre, email, ciudad)
VALUES (51, 'Ana', 'ana@mail.com', 'Madrid');  -- ✅ Correcto

-- Esta inserción falla porque ciudad no es Madrid
INSERT INTO v_clientes_madrid_v2 (id_cliente, nombre, email, ciudad)
VALUES (52, 'Luis', 'luis@mail.com', 'Barcelona'); -- ❌ Fallo CHECK OPTION
```

Efecto CASCADED en vistas anidadas (con nuestras tablas)

```
-- Vista base
CREATE VIEW v_base_madrid AS
SELECT id_cliente, nombre, ciudad
FROM clientes
WHERE ciudad = 'Madrid'
WITH CHECK OPTION CASCADE;

-- Vista derivada que además filtra por nombre que empiece con 'A'
CREATE VIEW v_nombre_a AS
SELECT id_cliente, nombre, ciudad
FROM v_base_madrid
WHERE nombre LIKE 'A%'
WITH CHECK OPTION CASCADE;

-- Intento de insertar un cliente con nombre 'Ana' pero ciudad 'Barcelona' (falla)
INSERT INTO v_nombre_a (id_cliente, nombre, ciudad)
VALUES (60, 'Ana', 'Barcelona');  -- ❌ No cumple ciudad='Madrid' de la vista base

-- Correcto: cumple ambas condiciones
INSERT INTO v_nombre_a (id_cliente, nombre, ciudad)
VALUES (61, 'Ana', 'Madrid');     -- ✅
```

Con `CASCADED` se verifican todas las condiciones de la cadena de vistas que tengan CHECK OPTION, ofreciendo máxima integridad.

📊 Vistas complejas: para reportes y análisis

Las vistas complejas (con JOIN, GROUP BY, funciones agregadas) son ideales para consultas de resumen pero **no son actualizables**. Intentar modificarlas produce error. Usando nuestro modelo:

Ejemplo: vista de total de ventas por cliente

```
CREATE VIEW v_ventas_por_cliente AS
SELECT c.id_cliente, c.nombre, c.apellido, SUM(v.cantidad) AS total_productos
FROM clientes c
JOIN ventas v ON c.id_cliente = v.cliente_id
GROUP BY c.id_cliente, c.nombre, c.apellido;

-- Cualquier UPDATE/INSERT sobre esta vista fallará
UPDATE v_ventas_por_cliente SET total_productos = 10 WHERE id_cliente = 3;  -- ❌ Error
```

Este tipo de vista solo debe usarse para consultas `SELECT`. Es una excelente herramienta para simplificar reportes complejos.

✏️ Mantenimiento de vistas

- **Reemplazar definición:** `CREATE OR REPLACE VIEW ...` (no es un ALTER, pero sobreescribe).
- **Eliminar:** `DROP VIEW nombre_vista;`

```
-- Reemplazar vista de clientes activos para incluir también la ciudad
CREATE OR REPLACE VIEW v_clientes_activos AS
SELECT id_cliente, nombre, apellido, email, ciudad
FROM clientes
WHERE activo = TRUE;

-- Eliminar vista obsoleta
DROP VIEW IF EXISTS v_ventas_por_cliente;
```

⚡ Consideraciones de rendimiento y buenas prácticas

- **Optimización:** las vistas complejas pueden ser lentas si las tablas base tienen millones de filas. Usa `EXPLAIN SELECT ... FROM vista` para analizar el plan de ejecución.
- **Índices:** las vistas no tienen índices propios; asegúrate de que las tablas base tengan índices adecuados para las columnas usadas en `WHERE` y `JOIN`.
- **Vistas anidadas:** pueden confundir al optimizador. Prefiere vistas planas o evalúa si es mejor una tabla temporal.
- **Seguridad:** una vista no oculta los datos si el usuario tiene permisos directos sobre las tablas base. Úsala junto con privilegios adecuados (conceder acceso solo a vistas).
- **Depuración:** cuando una vista no devuelve lo esperado, prueba ejecutar la consulta `SELECT` interna por separado para localizar el problema.

Consejo final: Las vistas son una herramienta poderosa, pero no abusar de ellas. En aplicaciones con alta carga transaccional, evalúa el costo de recalcular la vista cada vez. Para datos estáticos o reportes pesados, considera la simulación de vistas materializadas.

Este documento integra teoría, ejemplos prácticos con un modelo unificado y aclaraciones sobre conceptos clave como actualizabilidad, literales y vistas materializadas.
Con esta guía, un estudiante puede comprender a fondo las vistas en MySQL sin necesidad de consultar fuentes externas.